

ESP32 steuert Shelly Plus 1

Schalten per WLAN & HTTP-RPC — Arduino (C++) und MicroPython, inklusive Taster- und Zeitsteuerung

1. Grundprinzip

Der Shelly Plus 1 (Gen2) besitzt einen eingebauten Webserver mit einer RPC-Schnittstelle. Der ESP32 sendet dazu einfach einen HTTP-Request an die IP-Adresse des Shellys im lokalen Netzwerk:

```
http://<SHELLY-IP>/rpc/Switch.Set?id=0&on=true    -> einschalten  
http://<SHELLY-IP>/rpc/Switch.Set?id=0&on=false   -> ausschalten  
http://<SHELLY-IP>/rpc/Switch.GetStatus?id=0      -> Status abfragen
```

Voraussetzung: ESP32 und Shelly Plus 1 befinden sich im selben WLAN/Subnetz. Für eine zuverlässige IP-Adresse empfiehlt sich eine feste (statische) IP-Vergabe im Router oder in der Shelly-App.

2. Variante A: Arduino (C++)

Einfaches Beispiel mit der Arduino-IDE, das den Shelly abwechselnd ein- und ausschaltet.

Datei: esp32_shelly.ino

```

#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid = "DEIN_WLAN";
const char* password = "DEIN_PASSWORT";
const char* shellyIP = "192.168.1.50"; // IP deines Shelly Plus 1

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi verbunden");
}

void shellySwitch(bool state) {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    String url = "http://" + String(shellyIP) +
      "/rpc/Switch.Set?id=0&on=" + (state ? "true" : "false");

    http.begin(url);
    int httpCode = http.GET();

    if (httpCode > 0) {
      Serial.println("Antwort: " + http.getString());
    } else {
      Serial.println("Fehler: " + http.errorToString(httpCode));
    }
    http.end();
  }
}

void loop() {
  shellySwitch(true); // einschalten
  delay(5000);
  shellySwitch(false); // ausschalten
  delay(5000);
}

```

3. Variante B: MicroPython (Basis)

Gleiches Prinzip, umgesetzt in MicroPython mit der Bibliothek urequests.

Datei: main.py

```

import network
import urequests
import time

SSID = "DEIN_WLAN"
PASSWORD = "DEIN_PASSWORT"
SHELLY_IP = "192.168.1.50"

def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        print("Verbinde mit WLAN...")
        wlan.connect(SSID, PASSWORD)
        while not wlan.isconnected():
            time.sleep(0.5)
    print("Verbunden, IP:", wlan.ifconfig()[0])

def shelly_switch(state):
    url = "http://{}/rpc/Switch.Set?id=0&on={}".format(
        SHELLY_IP, "true" if state else "false"
    )
    try:
        response = urequests.get(url)
        print("Antwort:", response.text)
        response.close()
    except Exception as e:
        print("Fehler:", e)

connect_wifi()

while True:
    shelly_switch(True)
    time.sleep(5)
    shelly_switch(False)
    time.sleep(5)

```

Falls urequests fehlt, per mip nachinstallieren:

```

import mip
mip.install("urequests")

```

4. Kombiniert: Taster + Zeitsteuerung (MicroPython)

Diese Version schaltet den Shelly automatisch in einem festen Zeitintervall *und* reagiert zusätzlich sofort auf einen physischen Taster am ESP32. Die Hauptschleife arbeitet nicht-blockierend (kein `delay()`-Stau), damit der Taster jederzeit reaktionsschnell bleibt.

4.1 Verdrahtung

- Taster zwischen GPIO 4 und GND anschließen.
- Kein externer Pull-Up-Widerstand nötig – wird intern per `Pin.PULL_UP` aktiviert.
- Taster gedrückt = Pin liegt auf 0 (LOW).

4.2 Code

Datei: `main.py`

```

import network
import urequests
import time
from machine import Pin

# --- WLAN- und Shelly-Zugangsdaten ---
SSID = "DEIN_WLAN"
PASSWORD = "DEIN_PASSWORT"
SHELLY_IP = "192.168.1.50"

# --- Taster an GPIO 4, gegen GND, interner Pull-Up ---
BUTTON_PIN = 4
button = Pin(BUTTON_PIN, Pin.IN, Pin.PULL_UP)

# --- Zeitsteuerung: alle 60 Sekunden automatisch umschalten ---
AUTO_INTERVAL = 60 # Sekunden

relay_state = False
last_auto_switch = time.time()
last_button_state = 1
debounce_time = 0

def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        print("Verbinde mit WLAN...")
        wlan.connect(SSID, PASSWORD)
        while not wlan.isconnected():
            time.sleep(0.5)
    print("Verbunden, IP:", wlan.ifconfig()[0])

def shelly_switch(state):
    global relay_state
    url = "http://{}/rpc/Switch.Set?id=0&on={}".format(
        SHELLY_IP, "true" if state else "false"
    )
    try:
        response = urequests.get(url)
        response.close()
        relay_state = state
        print("Shelly geschaltet:", "AN" if state else "AUS")
    except Exception as e:
        print("Fehler:", e)

connect_wifi()

while True:
    now = time.ticks_ms()

    # --- Taster abfragen (mit einfachem Entprellen) ---
    if button.value() == 0 and last_button_state == 1:
        if time.ticks_diff(now, debounce_time) > 200:
            shelly_switch(not relay_state) # Zustand umschalten
            debounce_time = now
    last_button_state = button.value()

    # --- Zeitgesteuertes automatisches Schalten ---
    if time.time() - last_auto_switch >= AUTO_INTERVAL:
        shelly_switch(not relay_state)
        last_auto_switch = time.time()

    time.sleep_ms(20) # kurze Pause, haelt Taster reaktionsschnell

```

Anpassbar: AUTO_INTERVAL bestimmt den Zeittakt in Sekunden. Der Taster schaltet unabhängig davon jederzeit sofort um (Ein → Aus → Ein ...).

5. Wichtige Hinweise

Feste IP-Adresse: Statische IP im Router oder in der Shelly-Konfiguration vergeben, sonst funktioniert der Code nach einem Neustart des Shellys evtl. nicht mehr.

Gleiches Netzwerk: ESP32 und Shelly Plus 1 müssen sich im selben WLAN/Subnetz befinden.

RPC-Authentifizierung: Falls im Shelly ein Passwort für die RPC-API aktiviert ist, zusätzlich HTTP Basic-Auth im Request mitschicken (Base64-codiert).

Status abfragen: Mit `Switch.GetStatus` lässt sich zusätzlich der aktuelle Schaltzustand und der Stromverbrauch auslesen.

Alternative MQTT: Bei bereits vorhandenem MQTT-Broker (z.B. Home Assistant, Mosquitto) ist MQTT oft eleganter, da der Shelly dann auch Status-Änderungen aktiv meldet.

Beispiel für RPC-Authentifizierung in MicroPython:

```
import ubinascii

auth = ubinascii.b2a_base64("admin:passwort").decode().strip()
headers = {"Authorization": "Basic " + auth}
response = urequests.get(url, headers=headers)
```